

NAME: _____

CS3490 Midterm

For each problem, implement the functions satisfying the given specification.

You are free to define helper functions whenever you need to. You can also use built-in functions.

Don't copy the types of the functions.

Start writing the code at the left edge.

If necessary, you may abbreviate long function names to save space.

1. List Processing

1. Write a function `update :: (Eq a) => (a,b) -> [(a,b)] -> [(a,b)]` that takes a pair `(key,val) :: (a,b)` and a list `lst :: [(a,b)]` of key-value pairs.

`update (key,val) lst` returns the list `lst` but with the value associated to key `key` being replaced by `val`. If `lst` does not contain a pair whose first component is `key`, then the pair `(key,val)` should be added to the list.

```
> update (2,"Two") [(1,"One"),(2,"2")] = [(1,"One"),(2,"Two")]
> update (2,"Two") []                  = [(2,"Two")]
> update ("Y",True) [("X",True),("Y",False),("Z",False)]
    = [("X",True),("Y",True),("Z",False)]
```

2. `supset :: (Eq a) => [a] -> [a] -> Bool` takes lists `l1 l2`, and returns `True` if `l1` is a **superset** of `l2`: every element in `l2` is also present in `l1`.

```
> supset [1,2,3] [1,2] = True
> supset [5,4,3,2,1] [1,2,2,3] = True
> supset [] [1,2] = False
```

3. `findRoot :: (Integer -> Integer) -> [Integer] -> Maybe Integer` takes a function on integers and a list of them. It returns `Just x` if there is an element `x` that makes the function evaluate to 0, or else `Nothing`.

```
> findRoot (\x -> x^2 - 1) [0,1,2,3] = Just 1
> findRoot (\x -> x+1) [0,1,2,3] = Nothing
```

2. Combinators and Lambda Calculus

Recall the following λ -terms:

$$I = \lambda x.x$$

$$K = \lambda xy.x$$

$$S = \lambda xyz.xz(yz)$$

$$W = \lambda xy.xyy$$

Simplify the following term until a normal form with no redexes is reached, or a cycle is detected. You can use the rules of combinatory logic to take shortcuts with lambda reductions.

1. $S(WI)(SWI)K$

2. Let $T = \lambda pq.qIKSp$. Compute: $SIT(\lambda r.rI)$

3. Evaluate the following Haskell expression:
`flip (\x y -> x (x y)) 10 (3 *)`

3. Higher-order functions

Provide a *one-line implementation* for the following functions.

(If it's helpful, you may first write an implementation using pattern-matching and recursion. However, you will only receive credit for the one-line solution.)

1. `squashAll :: [String] -> [String]` takes a list of strings, and removes all the spaces from each string.

```
> squashAll ["hey there", "how are you?"] = ["heythere","howareyou?"]
```

```
> squashAll [" h o ", "ho", "h   o"] = ["ho","ho","ho"]
```

Hint. Use a combination of `map` and `filter`.

2. `concatPairs :: [(String,String)] -> [String]` takes a list of pairs of strings, and concatenates them componentwise.

```
> concatPairs [("Hey","You"),("Hello","World")] = ["HeyYou","HelloWorld"]
```

3. `noDups :: (Eq a) => [a] -> [a]` takes a list of elements that can be compared for equality, and removes all the duplicates from the list.

(The order in which elements are removed is not important.)

```
> noDups [5,3,6,4,7,5,7,6,8,5] = [5,3,6,4,7,8]
```

Hint. While there are many recursive solutions to this question, the easiest one to translate into a `fold` makes use of the `filter` function.

4. Tree functions

4.1. BTree

Consider the following datatype of binary+unary trees in which data occurs *at the leaves* as well as *at binary nodes* but *not at unary nodes*.

```
data BTree a = BLeaf a | UNode (BTree a) | BNode a (BTree a) (BTree a)
```

```
myTree :: BTree Integer
```

```
myTree = BNode 3 (UNode (BLeaf 5))  
          (BNode 4 (BLeaf 1) (UNode (BLeaf (-3))))
```

Implement the following functions.

1. `sumLeaves :: BTree Integer -> Integer` returns the sum of all data occurring in *the leaves* of the tree.

```
> sumLeaves myTree = 3
```

2. `mapBTree :: (a -> b) -> BTree a -> BTree b` applies the given function to all the data occurring in the tree.

```
> mapBTree (+10) myTree = BNode 13 (UNode (BLeaf 15))  
                              (BNode 14 (BLeaf 11) (UNode (BLeaf 7)))
```

3. `checkBNodes :: (Integer -> Bool) -> BTree Integer -> Bool` returns `True` if some data in the *binary nodes* of the tree satisfies the given predicate.

```
> checkBNodes (< 0) myTree = False  
> checkBNodes even myTree = True
```

5. Propositional logic

Recall the `Prop` datatype from the last homework:

```
type Vars = String
data Prop = Var Vars | Const Bool | And Prop Prop | Or Prop Prop | Not Prop
```

Define the following functions.

1. `occurs :: Vars -> Prop -> Bool` takes a variable `x :: Vars` and a formula `p :: Prop` and returns `True` if the variable `x` occurs in the formula, and returns `False` otherwise.

```
> occurs "X" (Or (Var "X") (Var "Y")) = True
> occurs "Z" (Or (Var "X") (Var "Y")) = False
```

2. `countBin :: Prop -> Integer` counts the number of *binary operators* occurring in the given formula.

```
> countBin (And (Var "X") (Var "Y")) = 1
> countBin (And (Not (Var "X")) (Or (Var "X") (Var "Z"))) = 2
```

3. `substConst :: Vars -> Bool -> Prop -> Prop` takes a variable `x`, a boolean constant `b`, and a formula `p`. It returns the formula `p` with *every* occurrence of the variable `x` replaced by `b`.

```
> substConst "X" True (Or (Var "X") (Var "Y")) = Or (Const True) (Var "Y")
> substConst "Z" True (Or (Var "X") (Var "Y")) = Or (Var "X") (Var "Y")
```

4. (**Extra credit 1.**) Recall the following classification of formulas in propositional logic.

- A formula is *satisfiable* if there *exists* a variable environment making the formula true.
- A formula is a *contradiction* if *no* variable environment makes the formula true.
- A formula is a *tautology* if *every* variable environment makes the formula true.
- A formula is a *contingency* if some environments make it true, and some make it false: it is *neither* a contradiction *nor* a tautology.

Assume you have implemented a function `sat :: Prop -> Bool` which outputs whether a formula is *satisfiable*.

Use the `sat` function as a helper to implement a function `tauto :: Prop -> Bool` which checks whether a formula is a tautology. Your solution should fit on one line.

(*Hint.* Think about the relationship between being satisfiable and being a tautology.)

```
> tauto (Or (Var "X") (Not (Var "X"))) = True
> tauto (Or (Var "X") (Not (Var "Y"))) = False
```

5. (**Extra credit 2.**) Consider the following generic fold for propositions.

```
foldProp :: (Vars -> b) -> (Bool -> b) -> (b -> b -> b) -> (b -> b -> b) -> (b -> b)
          -> Prop -> b
foldProp v c a o n (Var x)      = v x
foldProp v c a o n (Const y)   = c y
foldProp v c a o n (And p1 p2) = a (foldProp v c a o n p1) (foldProp v c a o n p2)
foldProp v c a o n (Or p1 p2)  = o (foldProp v c a o n p1) (foldProp v c a o n p2)
foldProp v c a o n (Not p)      = n (foldProp v c a o n p)
```

Implement the `occurs` function in one line using the above fold.